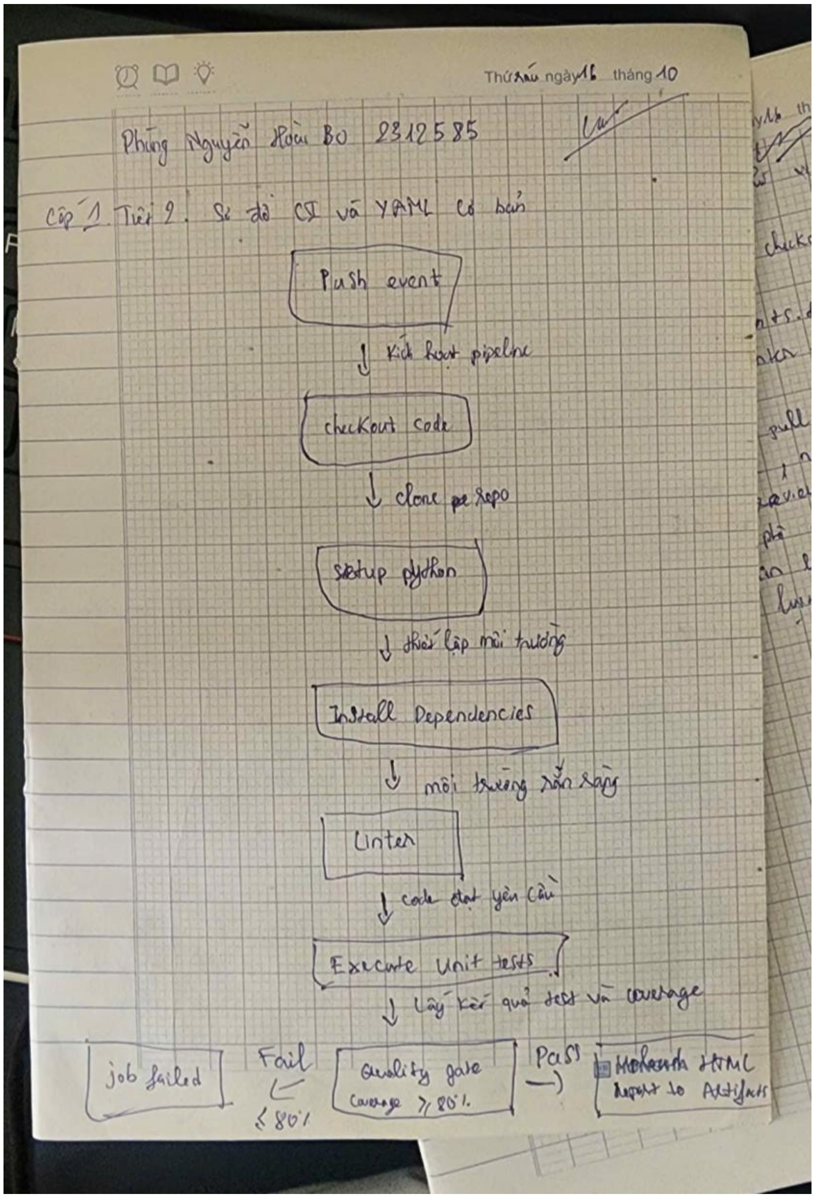


Thứ sáu ngày 16 tháng 10

Phùng Nguyễn Hoài Bảo 2312585

Cấp 1. Tiết 2. Sơ đồ CI và YAML cơ bản

Push event

↓ kích hoạt pipeline

checkout code

↓ clone repo

setup python

↓ thiết lập môi trường

Install Dependencies

↓ môi trường sẵn sàng

Linter

↓ code đạt yêu cầu

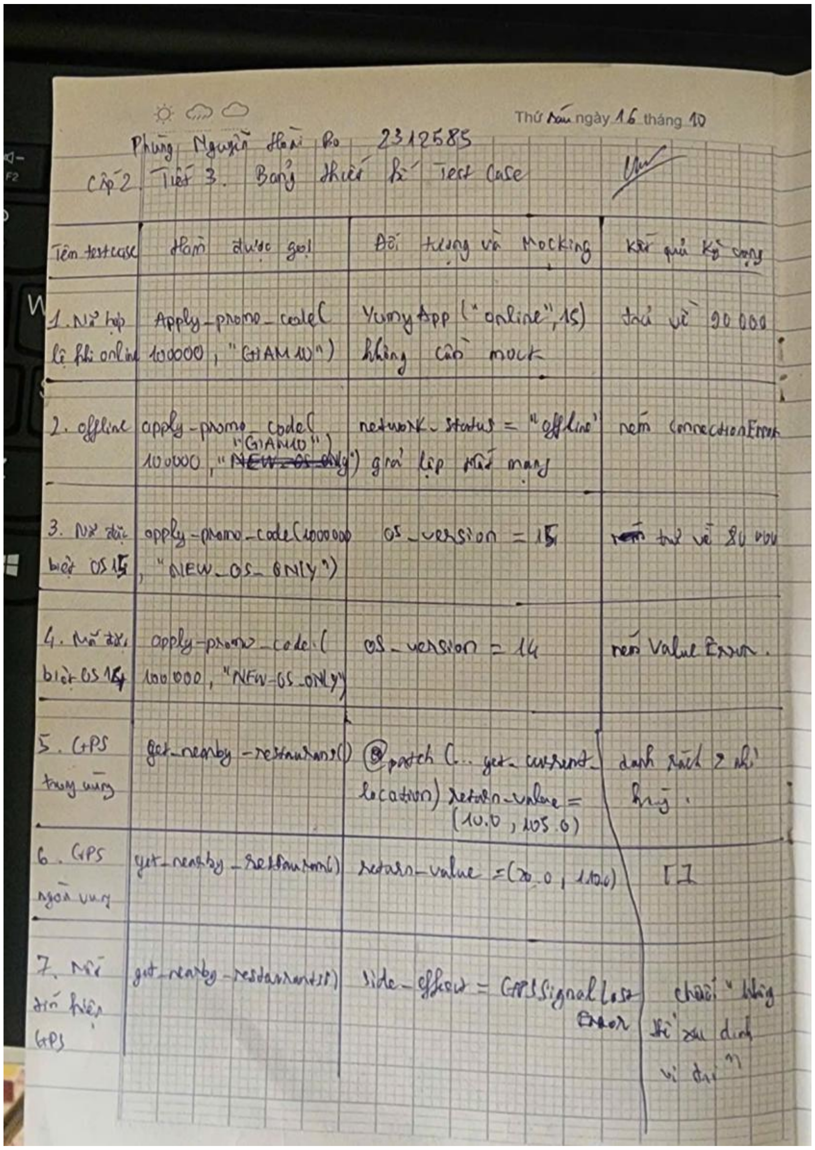
Execute Unit tests

↓ Lấy kết quả test và coverage

Quality gate coverage ≥ 80%

Fail ← < 80% job failed

Pass → Upload HTML report to Artifacts



Thứ Sáu ngày 16 tháng 10

Phùng Nguyễn Hoài Bảo 2312585

Cặp 2. Tiết 3. Bảng thiết kế Test Case

Tên testcase	Hàm được gọi	Đối tượng và Mocking	Kết quả kỳ vọng
1. Mã hợp lệ khi online	Apply_promo_code(100000, "GIAM10")	YumyApp("online", 15) không cần mock	trả về 90 000
2. offline	apply_promo_code(100000, ~~"NEW_OS_ONLY"~~ "GIAM10")	network_status = "offline" giả lập mất mạng	ném ConnectionError
3. Mã đặc biệt OS 15	apply_promo_code(1000000, "NEW_OS_ONLY")	os_version = 15	~~ném~~ trả về 80 000
4. Mã đặc biệt OS 14	apply_promo_code(100000, "NEW_OS_ONLY")	os_version = 14	ném ValueError.
5. GPS trong vùng	get_nearby_restaurants()	@patch(... get_current_location) return_value = (10.0, 105.0)	danh sách 2 nhà hàng.
6. GPS ngoài vùng	get_nearby_restaurants()	return_value = (20.0, 110.0)	[]
7. Mất tín hiệu GPS	get_nearby_restaurants()	side_effect = GPSSignalLostError	chuỗi "không thể xác định vị trí"

Phùng Nguyễn Khoa Bo 2312585

Cấp độ 2.

Câu 2.1. Tại sao bước chạy Linter (flake8) phải đặt trước bước chạy test trong đường ống CI?

- Linter (flake8) chỉ phân tích tĩnh, chạy vài giây, không cần thực thi code. Test và coverage chạy lâu hơn, tốn tài nguyên runner.
- Nếu code có lỗi cú pháp thì test chắc chắn không chạy nổi, chạy test chỉ phí thời gian.
- Đây là nguyên lý Fail-Fast: lỗi rẻ nhất thì kiểm tra trước, báo lỗi sớm nhất, giúp vòng phản hồi nhanh.

Câu 2.2 Tham số --fail-under=80 và lệnh coverage report hoạt động theo nguyên lí nào trong Linux để báo hiệu cho GitHub Action dừng tiến trình build?

- Mọi tiến trình Linux khi kết thúc đều trả về mã thoát (exit code) cho tiến trình cha. Mã 0 là thành công, khác 0 là thất bại.
- Khi coverage dưới 80%, coverage report --fail-under=80 kết thúc với mã khác 0. (coverage dùng mã 8) khi đạt ngưỡng thì trả về 0.
- Runner của github actions chạy script bằng bash với chế độ dừng khi gặp lỗi. Thấy mã khác 0 thì đánh dấu step là failed, bỏ qua các step sau (trừ step có if: always()) và job báo đỏ.

Thứ năm ngày 16 tháng 10

Phùng Nguyễn Thái Bo 2312585

~~- Bạn có thể minh họa~~ :

Câu 2.3. Tham số -m là gì giúp ích gì ?

- -m là missing : hiện cột Missing liệt kê số dòng code chưa được test chạy tới.
- Lợi ích : biết chính xác dòng nào chưa được phủ thay vì chỉ thấy phần trăm, nên biết viết test cho hàm hoặc nhánh nào, tối ưu cho code hơn.

Câu 3.1. Flaky Test và lỗi sai lệch môi trường

- Flaky test : cùng một code, cùng một test nhưng lúc pass lúc fail, không có thay đổi nào. Nguyên nhân là yếu tố không xác định : đồng hồ thật, thời gian, thứ tự chạy, mạng.
- Lỗi sai lệch môi trường phụ thuộc : test fail ổn định, lặp lại được, vì môi trường khác nhau.

Câu 3.2. Vì sao phải ghim phiên bản trên CI ?

- Runner CI là máy sạch, mỗi lần chạy cài lại từ đầu. Nếu ghi requests không có số phiên bản, pip lấy bản mới nhất, nếu hôm nay xanh, ngày mai thư viện ra bản mới có thể đỏ dù code không đổi.
- Ghim (== 2.31.0) đảm bảo tái lập : máy mọi người và CI chạy đúng cùng một bản, dễ truy vết lỗi.
- Ngăn ra thành breaking change bất ngờ và khi so bản mới thì bạn chủ động kiểm soát.

Thứ năm ngày 16 tháng 10

Phùng Nguyễn Hoài Bảo 2312586

Câu 5.3 Quy trình 6 bước nâng cấp thư viện qua pull request.

1. Tạo nhánh riêng từ main: git checkout -b chore/upgrade-request.
2. Nâng số phiên bản trong requirements.txt
3. Cài lại và chạy toàn bộ linter, test, coverage trên máy.
4. Commit, push nhánh và mở pull request vào main
5. CI chạy tự động trên PR, nếu đỏ thì sửa trên nhánh, đồng thời đợi review.
6. Khi CI xanh và được phê duyệt thì merge vào main, rồi xoá nhánh main luôn ổn định vì thay đổi đã qua cổng chất lượng.

HONG HA